FeatureScalpel: Transient Sparse-Autoencoder Edits and Their Transfer Between Language Models

A Main Project Report

Submitted to the APJ Abdul Kalam Technological University
in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering(AI)

by

ADITHYAN R S *(B23CS2109)*
ALONA BIJU *(B23CS2113)*
ALVIN BINU *(B23CS2114)*
DARSAN M *(B23CS2125)*

Department of Computer Science and Engineering
Mar Baselios College of Engineering and Technology
(Autonomous)
Mar Ivanios Vidya Nagar, Nalanchira
Thiruvananthapuram–695015

October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MAR BASELIOS COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
MAR IVANIOS VIDYA NAGAR, NALANCHIRA
THIRUVANANTHAPURAM–695015

CERTIFICATE

This is to certify that the report entitled **FeatureScalpel: Transient Sparse-Autoencoder Edits and Their Transfer Between Language Models** submitted by **Adithyan R S** (KTU Reg No: B23CS2109), **Alona Biju** (KTU Reg No: B23CS2113), **Alvin Binu** (KTU Reg No: B23CS2114) and **Darsan M** (KTU Reg No: B23CS2125) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering (AI) is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Poorna B R
(Project Guide)
[TODO: designation]
Department of CSE

Dr. Jisha John
(Head of the Department)
Professor
Department of CSE

Place: Thiruvananthapuram Date:

Declaration

We hereby declare that the project report entitled **FeatureScalpel: Transient Sparse-Autoencoder Edits and Their Transfer Between Language Models**, submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI), is a record of original work carried out by us at the Department of Computer Science and Engineering, Mar Baselios College of Engineering and Technology (Autonomous), Thiruvananthapuram. The experimental results reported here were produced by the system described in this report. Wherever the work of others has been used, it has been duly acknowledged and cited. This report has not been submitted, in whole or in part, to any other University or Institute for the award of any degree or diploma.

Place: Thiruvananthapuram
Date:

Adithyan R S (B23CS2109)
Alona Biju (B23CS2113)
Alvin Binu (B23CS2114)
Darsan M (B23CS2125)

Acknowledgement

We express our deepest gratitude to our **Project Guide** from the Department of Computer Science and Engineering for continuous guidance, critical feedback and technical insight throughout this project. Their advice on experimental rigour and on stating results precisely shaped both the direction and the quality of this work.

We sincerely thank **Dr. Jisha John**, Head of the Department of Computer Science and Engineering, for academic support, provision of resources and encouragement at every stage of the project.

We acknowledge the open-source communities behind **PyTorch**, **TransformerLens**, **SAELens**, **Hugging Face** and **Streamlit**, the author of the pretrained **gpt2-small-res-jb** sparse autoencoders, and the **Neuronpedia** project. We also thank the authors of ROME, whose openly released fact datasets, including CounterFact, formed the basis of the benchmarks, and the authors of the model-stitching work on which our translators build.

Finally, we are grateful to our families and friends for their patience and encouragement throughout this endeavour.

Adithyan R S
Alona Biju
Alvin Binu
Darsan M

Abstract

Language models sometimes rank a correct completion below competing tokens. Fixing this by changing weights is permanent and degrades as edits accumulate; fixing it by adding text to the prompt does not use what the model already computes. This project, **FeatureScalpel**, studies a **transient, weight-free alternative**: during one forward pass the residual stream of GPT-2 Small is edited through the features of a pretrained **sparse autoencoder (SAE)**, and the edit disappears when the pass ends. Per-feature gradient descent on the multipliers of 200 candidate features brought the true answer to rank 1 on 271 of 300 held-out CounterFact prompts, against 161 for a sweep given the same range and 94 for the original sweep settings. Two limits of that work motivated the second part of the project: every edit needs an SAE for its own model, and reaching rank 1 by steering proved weak evidence, since an additive edit forced random unrelated words to rank 1 on 20 of 20 prompts.

The second part asks whether the change an edit makes inside one model can be carried into another at inference time, with no weight change. A linear map between the residual streams of GPT-2 Small and GPT-2 Medium keeps the receiving model working (per-dimension R^2 0.58 and 0.66) and carries a known meaning change far above controls (55% and 33%). An edit tuned only in Small, translated and added to Medium (*Export*), moves the target's median rank from 142 to 14 but makes it the top answer on only 8% of 88 test records. Seven further attempts at a better translator, injection layer, training signal or amount of training data stopped at about 20% at best, whereas tuning the edit against Medium's output for each sentence reached 95%, a ceiling and not a transfer. The effect on reworded prompts is the same for a random target word as for the true one, so what carries is a push toward the targeted word, not a fact. In the other direction (*Import*), replacing Small's state by Medium's translated state rescues one in three of the facts Small gets wrong but lowers its overall accuracy, whereas a half-and-half mix raises accuracy on held-out facts from 8.5% to 9.5% (Medium alone: 10.2%). Small's SAE directions raise the steering success from 81% to 95% over random directions but are not required for it.

All pass bars were fixed before the runs, every result is stored with its logs, and the limits are stated: one model pair, one dataset, 88 test records, no generated-text or other-model tests.

Table of Contents

Acknowledgement i

Abstract ii

List of Figures v

List of Tables vii

1 Introduction 1

1.1 Background and Motivation 1
1.2 Problem Statement and Research Questions 2
1.3 Objectives and Scope 3
1.4 Contributions 3
1.5 Organisation of the Report 4

2 Literature Review 5

2.1 Knowledge Editing 5
2.2 Activation Steering and Sparse Autoencoders 5
2.3 Comparing and Stitching Representations 6
2.4 Research Gap 6

3 Methodology 7

3.1 Models, Data and Evaluation Records 7
3.2 The Transient Edit in One Model 7
3.3 Translators 8
3.4 Export and Import 9
3.5 Metrics, Controls and Statistics 9

4 Implementation 11

4.1 Software, Hardware and Code Organisation 11
4.2 Reproducibility 11
4.3 Engineering Notes 11

4.4 Development Process 12

5 Results and Discussion **13**

5.1 The Transient Edit in One Model 13

5.2 The Channel: Translator Quality and a Positive Control 14

5.3 Export with the Text-Fitted Map 15

5.4 Searching for a Better Translator 15

5.5 What Is Carried: a Fact or a Push? 18

5.6 Does Small's SAE Matter for Steering Medium? 19

5.7 Import: Medium's State into Small 19

5.8 Discussion and Threats to Validity 21

6 Conclusion and Future Work **22**

6.1 Conclusion 22

6.2 Future Work 23

References **24**

Appendices **28**

Appendix A: Pass Bars and Verdicts 28

Appendix B: Reproducing the Results 29

List of Figures

1.1 The two directions studied. A translator maps the residual stream of one model into the other's coordinates; no weight changes in either model, and the SAE is used only where the edit is made. 2

3.1 Data pipeline of the benchmark (first part of the project): CounterFact, the hard set of prompts whose true answer starts at rank 2 to 1000, and a subject-safe split with held-out relations. 8

5.1 Prompts brought to rank 1 (labels give the count) by starting rank of the true answer, GPT-2 Small layer 8, 300 held-out CounterFact prompts. Gradient descent is the green bars. 14
5.2 An additive edit forces random words to rank 1 (a) at sizes comparable to or larger than the residual stream itself (b); the multiplier edit does not. 15
5.3 Country-swap control: the translated difference moves the receiver far above the best control (tick). 16
5.4 Export of the edit tuned in small, small layer 8 to medium layer 16, 88 test records: rank (a) and top-1 rate (b) against the dose. The word push uses the answer and is the comparison, not a method. 16
5.5 What the export does to reworded prompts and to neighbouring facts. Grey bars are references. The word push and the fact in the prompt damage neighbours far more than the translated edit. 17
5.6 Top-1 of every export variant on the same 88 test records. Blue: map fitted on text; green: map trained on edits; red: per-sentence tuning against medium's output (a ceiling, not a transfer). 18
5.7 More and more varied training edits: top-1 on the map's own training prompts and on unseen test records (3 seeds, dots). The gap closes, but the test result does not rise. 19
5.8 Gain on reworded prompts, each word against its own unchanged level: the real target and a random word gain about the same. 19
5.9 The tuning of Figure 5.6, step D6, with small's SAE directions and with 200 random directions (88 test records). 20

5.10 Import by mixing small's own state with medium's translated state. (a) What the mix keeps and fixes. (b) Net accuracy: best at a 50/50 mix, below small alone under full replacement. 21

List of Tables

5.1 Single-model editing on 300 held-out CounterFact prompts. 14
5.2 Ridge translators for the two directions studied (wikitext-2, held-out tokens). 15
5.3 Export of the edit through the ridge map (88 primary test records; dose chosen on dev). Unchanged medium: median rank 142. 17
5.4 The search for a better translator (88 primary test records, same edits, controls and judging throughout). 18
5.5 Import on the held-out half B of the facts (10,895 facts): replacement and the 50/50 mix against the two models alone. 20

A.1 Pass bars (“gates”) and their verdicts. 28

Chapter 1

Introduction

1.1 Background and Motivation

Large language models sometimes complete a factual prompt with a generic word while the correct answer sits a few positions lower in the output ranking. For "The location of Massachusetts Institute of Technology is in", GPT-2 Small [1] may rank " the" first and " Cambridge" lower. We call such cases **suppressed factual recall**; the term describes the output distribution only and makes no claim about what the model "knows". The usual remedies change the model: fine-tuning, or closed-form weight edits such as ROME [2] and MEMIT [3], which are permanent and, when applied repeatedly, first erase earlier edits gradually and then collapse the model [4]. Retrieval and in-context editing [30, 27] leave the weights alone but supply the fact as extra text.

Mechanistic interpretability offers a third route. A sparse autoencoder (SAE) [6, 7] re-expresses a layer's activations in a much wider, sparsely active basis whose directions, called features, often match single concepts. An SAE decoder direction is a vector in the residual stream, so a feature can be turned up or down during one forward pass and discarded afterwards. The first part of this project, **FeatureScalpel**, built such a *transient edit* for GPT-2 Small: for a prompt whose true answer is outranked, one multiplier per candidate SAE feature is tuned by gradient descent so that the answer becomes the top prediction, with the model's weights untouched (Section 5.1). On 300 held-out CounterFact prompts it reached rank 1 on 271, against 161 for the best fixed sweep.

That work left three open problems, and together they led to the second part of the project:

1. **An edit is tied to one model with an SAE.** Trained SAEs exist only for some models and layers, so an edit made where the tooling exists cannot simply be reused elsewhere.
2. **GPT-2 Small is weak.** It often lacks the true answer, which larger models of the same family hold; the interesting direction may be a stronger model helping a weaker one.
3. **Rank 1 is a weak test.** An additive edit that switches on silent features pushed random

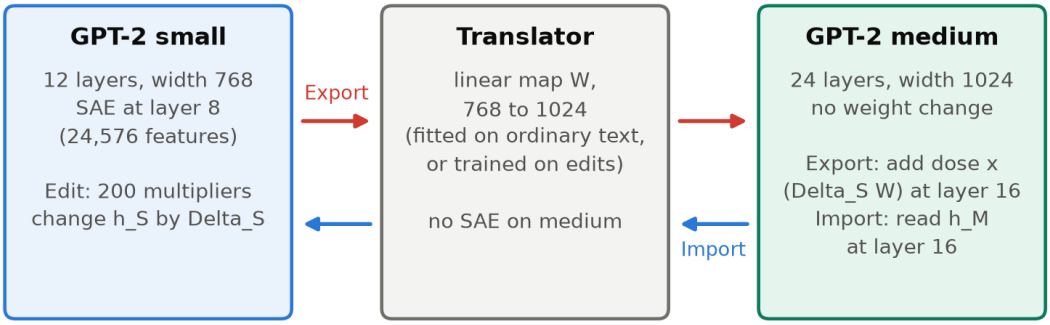


Figure 1.1: The two directions studied. A translator maps the residual stream of one model into the other's coordinates; no weight changes in either model, and the SAE is used only where the edit is made.

unrelated words to rank 1 on 20 of 20 prompts (Section 5.1). Success under steering therefore has to be judged against controls that separate a fact from a push toward a word.

These suggest a question: *can the change an edit makes inside one model be carried into another model at inference time, with no weight change, and what exactly is carried?* If it can, an edit could be made once, where an SAE and tools exist, and applied elsewhere (**Export**); and a stronger model's internal state could improve a weaker one without any training (**Import**). Figure 1.1 shows both directions for GPT-2 Small and GPT-2 Medium, the pair studied here.

1.2 Problem Statement and Research Questions

Given an edit tuned inside GPT-2 Small, can a map between the two models' residual streams carry its effect into GPT-2 Medium (and the reverse), specifically, with limited side effects, and without the receiver being told the answer? The work is organised around five research questions.

1. **RQ1 (channel).** Can a linear map between the residual streams keep the receiving model working and carry a known change of meaning?
2. **RQ2 (export).** Does an edit tuned only in Small move Medium's answer toward the target, how specifically, and which kind of translator is best?
3. **RQ3 (limits).** What limits Export: the injection layer, the translator's capacity, the amount or variety of training edits, or the training signal?
4. **RQ4 (import).** Can Medium's state improve Small's answers without telling Small the answer?

5. **RQ5 (what is carried).** Is what crosses a fact or a push toward a word, and does Small's SAE matter for steering Medium?

1.3 Objectives and Scope

The objectives are to build a translator between two models and test it with a positive control; to measure Export and Import on a held-out benchmark with controls, paired statistics and pass bars fixed before each run; to search systematically for a better translator and record every failure with its evidence; and to keep every result reproducible. The study uses GPT-2 Small (12 layers, width 768, SAE `gpt2-small-res-jb` at layer 8) and GPT-2 Medium (24 layers, width 1024, no SAE, injection at layer 16), single-token CounterFact [2] facts and next-token measurements. The additive edit, generated text, other model pairs and relations, and comparison with ROME and MEMIT are outside the scope and are listed as untested.

1.4 Contributions

1. **A transient SAE edit with an equal-budget comparison.** Per-feature gradient descent beats a sweep given the same range (271 against 161 of 300 prompts), and an additive variant is shown to be an unreliable measure of success (Section 5.1).
2. **A translator with a positive control and a controlled Export test.** A linear map fitted on ordinary text keeps the receiver working and carries a country swap at 55% and 33% against controls under 0.5%; an edit tuned in Small lifts the target's rank in Medium from 142 to 14 specifically, but rarely to rank 1 (Sections 5.2 and 5.3).
3. **A documented search that locates the limit.** Seven attempts to improve the export stop at about 20% top-1 at best. The limit is not the injection layer, the translator's capacity, a missing rank-1 term in the loss, memorisation of training words or the variety of target words, and per-sentence tuning (95%) shows that the channel itself can carry a rank-1 edit (Section 5.4).
4. **Evidence about what is carried.** The effect on reworded prompts is the same for a random word as for the true target, so Export transfers a push, not a fact (Section 5.5).
5. **A first positive Import result.** Mixing half of Medium's translated state into Small raises its accuracy on held-out facts from 8.5% to 9.5% while keeping 98% of the facts both models knew (Section 5.7).

1.5 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 describes the edit, the translators, Export and Import, and the evaluation protocol. Chapter 4 gives the implementation. Chapter 5 reports the experiments in the order in which they were done and discusses what they show and do not show. Chapter 6 concludes. The appendices list every pass bar with its verdict and the commands to reproduce the results.

Chapter 2

Literature Review

This chapter reviews the four research threads on which the project draws and states the gap it addresses.

2.1 Knowledge Editing

ROME [2] locates a factual association in the mid-layer MLPs of GPT-style models and rewrites one weight matrix in closed form; MEMIT [3] extends this to thousands of edits. Both change the weights permanently. Gupta *et al.* [4] showed that sequential ROME and MEMIT edits first erase earlier edits gradually and then collapse the model abruptly, and Hase *et al.* [26] found that where a fact is localised need not predict where it can be edited best. Methods that leave the weights alone add the fact as text: in-context editing [27] places the corrected fact and demonstrations in the prompt, and retrieval-augmented generation [30] conditions the model on retrieved documents. Both change the input and not the model's internal computation. EasyEdit [28] collects these methods in one framework. The CounterFact dataset of Meng *et al.* [2], which pairs a prompt with a true and a counterfactual answer, is the benchmark used throughout this report.

2.2 Activation Steering and Sparse Autoencoders

Activation steering adds a direction to the residual stream at inference time [9, 10, 11]. Superposition [5] means that single neurons rarely correspond to single concepts; sparse autoencoders [6, 7, 8] learn a wide sparse basis whose features are often interpretable, and open SAEs now exist for GPT-2 Small [15], for Gemma [16] and others, with browsable feature explanations [17]. Several studies edit SAE features directly: Chalnev *et al.* [24] target SAE features to limit side effects of steering vectors, Zhao *et al.* [25] edit a few features to control whether a model relies on its memory or on the context, Farrell *et al.* [23] use feature ablation to unlearn knowledge, Marks *et al.* [29] use features as nodes of editable circuits, and Arad *et al.*

[19] show that selecting features by their effect on the output improves steering. The evidence is mixed: AxBench [12] reports that simple baselines can beat SAEs for steering, a Google DeepMind update [20] reports negative results for SAEs on downstream tasks, and Chaudhary and Geiger [21] and Cui *et al.* [22] report limits when SAEs are used to disentangle or to intervene on factual knowledge. Steering that reaches a target token is also known to be weak evidence about what a model contains, which is why this project uses controls throughout.

2.3 Comparing and Stitching Representations

Model stitching connects the lower layers of one network to the upper layers of another through a small learned layer; if the stitched network still works, the two representations are compatible [34, 35]. Representational similarity measures such as CKA [36] compare activations without stitching. For language models, Chen *et al.* [37] fit affine maps between the residual streams of two models, including GPT-2 Small and GPT-2 Medium, and use them to transfer SAEs, probes and steering vectors from a small model to a larger one. [TODO: this description of [37] was written from the paper's summary; check it against the full text before submission] Our translator is of this family: a ridge regression [38] fitted on ordinary text. We do not claim the map as new.

2.4 Research Gap

Knowledge editing changes weights or the prompt; SAE steering works inside one model; stitching work transfers vectors learned in one model, such as SAEs, probes and steering directions. To our knowledge, based on a limited search, no work has asked whether the change that a fact edit makes inside one model can be carried into another, in both directions, with controls that separate a carried fact from a push toward a word, and with every pass bar fixed before the run. This report supplies that test for GPT-2 Small and Medium, including the negative results.

Chapter 3

Methodology

3.1 Models, Data and Evaluation Records

Models. GPT-2 Small ("small": 12 blocks, width 768) [1] with the pretrained SAE `gpt2-small-res-jb` (24,576 features at `blocks.8.hook_resid_pre`) [15], and GPT-2 Medium ("medium": 24 blocks, width 1024), which has no SAE. Both share one tokenizer, so token i of a sentence is the same token in both models. All maps act on `hook_resid_pre` of a chosen block; small's layer 8 is paired with medium's layer 16 (both two thirds of the way through their stacks).

Data. The CounterFact hard set of 16,360 prompts and a subject-safe split (Figure 3.1) were built in the first part of the project; the transfer study uses a 200-record benchmark drawn from it, split into 50 development (dev) and 150 test records. Dev records choose settings (layer, dose, epoch); test records are only read. The edit in small reaches rank 1 on 116 of the 200 records; the **88 primary test records** are those test records on which it does, since only there is there an edit to transfer. Medium's median rank of the target before anything is done is 142. Training edits for learned maps come from a separate pool of 1,450 prompts, from which every prompt whose subject or target word belongs to a dev or test record is removed. Import is evaluated on the 21,789 single-token CounterFact facts that remain after the 130 development and pilot records are excluded.

3.2 The Transient Edit in One Model

At layer 8 the SAE encodes the residual stream x_p at position p into feature activations $f_{p,k}$. The **multiplier edit** rescales feature k by a multiplier m_k and adds only the difference, so the SAE's reconstruction error is never injected:

$$\Delta_p \;=\; \sum_{k \in \mathscr{K}} (m_k - 1)\, f_{p,k}\, W_{\text{dec}}[k], \qquad x'_p = x_p + \Delta_p, \tag{3.1}$$

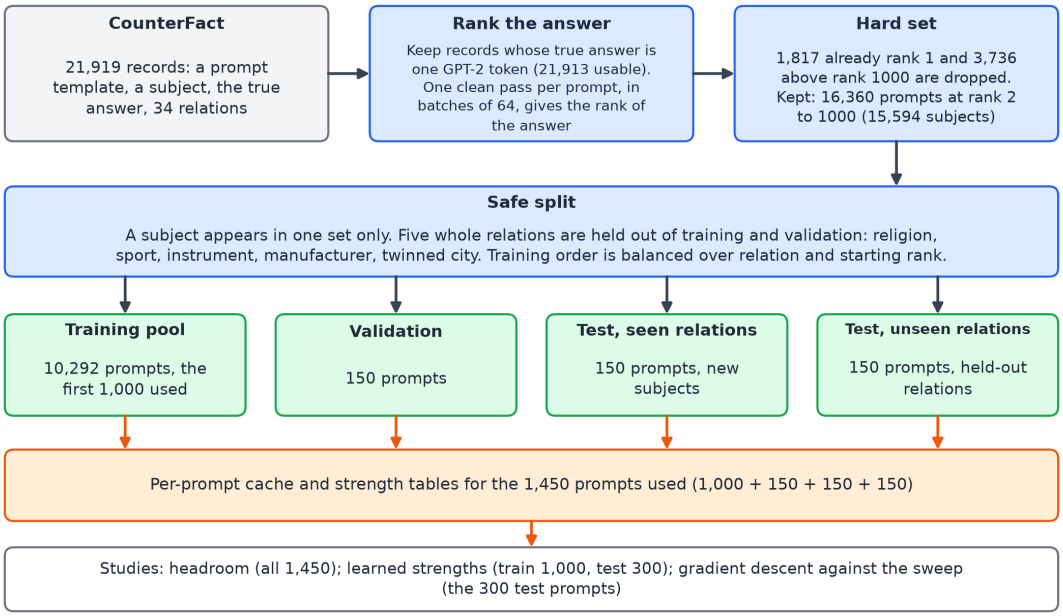


Figure 3.1: Data pipeline of the benchmark (first part of the project): CounterFact, the hard set of prompts whose true answer starts at rank 2 to 1000, and a subject-safe split with held-out relations.

where $\mathscr{K}$ is a set of 200 candidate features chosen by a causal ranking of the active features, $W_{\text{dec}}[k]$ is the decoder direction of feature k, and the edit acts at every position ("relay mode": the same multipliers can be re-applied to any other sentence by feature id). Gradient descent tunes $m_k = 3\sigma(z_k) \in (0,3)$ with Adam [31] (learning rate 0.1, 100 steps) on the loss

$$\mathscr{L} \;=\; \max\!\big(0,\, \gamma - (\ell_t - \max_{v \neq t} \ell_v)\big) \;+\; \lambda_{\text{KL}}\, \text{KL}\, \mathbf{1}[\text{target leads}] \;+\; \lambda_{\text{size}} \sum_k |m_k - 1|\, w_k, \qquad (3.2)$$

with margin $\gamma = 0.3$, $\lambda_{\text{KL}} = 3$, $\lambda_{\text{size}} = 0.005$, where ℓ are the output logits, t the target and the KL term measures the change of the rest of the next-word distribution. Only the multipliers change; the model and the SAE are frozen.

3.3 Translators

A translator maps a state of one model into the coordinates of the other. Only the *change* of an edit is translated, so the bias of an affine map cancels and the translated change is $\Delta_S W$ (zero at the first position). Four ways of obtaining W were tried, from simplest to most elaborate.

1. **Ridge map fitted on ordinary text** (the baseline): $W = \arg\min \|(H_S - \mu_S)W - (H_M - \mu_M)\|^2 + \lambda \|W\|^2$ on paired states of the two models over wikitext-2 (500,126 fit tokens, 100,076 held-out) [38]. Maps were fitted in both directions for 25 layer pairs.

2. **Neural map fitted on text**: the ridge map plus a one-hidden-layer correction with a

GELU non-linearity, trained from the ridge solution.

3. **Linear map trained on edits (output matching).** Starting at the ridge map, W is trained so that medium's answers move the way small's answers move under an edit. With ℓ_M medium's clean logits and $\ell_S^{\text{edit}} - \ell_S$ the shift small's edit makes, the goal is $q = \text{softmax}(\ell_M + \ell_S^{\text{edit}} - \ell_S)$ and the loss is $\text{KL}\big(q \,\|\, p_M(h_M + \alpha \Delta_S W)\big) + \beta \|W - W_0\|^2 / \|W_0\|^2$, with training texts (the tuned prompt, two rewordings, two neighbours) from 289 successful training edits. Variants: more epochs, a rank-1 hinge term on the loss, and 1,221 additional edits toward random words.

4. **No fixed map: per-sentence re-tuning (a ceiling).** The 200 multipliers are tuned by the loss of Equation 3.2 but measured on *medium's* output after the translated change is injected. This looks at medium for every sentence and is therefore not a transfer; it shows what the channel can carry.

3.4 Export and Import

Export. While medium reads a sentence, the translated change is added at layer 16 at every position but the first: $h'_M = h_M + \alpha \Delta_S W$. The scalar α, the **dose**, is chosen on the dev records (from step D10 on by the best mean log-rank gain of the target, because choosing by dev top-1 on 28 records proved fragile, Section 5.4). The receiver is never shown the answer.

Import. Small's layer-8 state is mixed with medium's translated layer-16 state: $h'_S = (1-\gamma)\, h_S + \gamma T(h_M)$ at every position but the first, with $T(h) = (h - \mu_M) W_{M \to S} + \mu_S$; $\gamma = 1$ is full replacement. Small's remaining four layers then run unchanged, and small is never told the answer.

3.5 Metrics, Controls and Statistics

Metrics. *Top-1*: the target is the model's first choice; *rank* of the target and the *log-rank gain* over the unchanged rank; *rewordings* (PS): the share of reworded prompts on which the new answer beats the true one; *neighbours* (NS): the share of prompts about other subjects of the same relation on which the true answer still beats the new one (higher is fewer leaks); *unrelated prompts*: KL divergence and the share of top-1 changes on 12 neutral prompts.

Controls. A *random vector* of the same size at every position; *another record's edit*; an edit toward a *random word* of matched starting rank, judged on its own word; a *word push* along the target's output direction (uses the answer; not required to be beaten); the *fact stated in medium's prompt*; for Import, a state from a wrong sentence, the average state and a random map.

Statistics and discipline. Paired one-sided Wilcoxon tests [33] on per-record outcomes, exact McNemar-type tests [32] for paired right/wrong facts, bootstrap intervals, and seeds

reported with ranges. **Every pass bar ("gate") was written into the project plan before the run that judged it and never moved**; steps designed after seeing a result are labelled post-hoc and cannot change a verdict. The bar for a working export is 50% top-1; the full list is in Appendix A. Everything chosen on dev (layer, dose, epoch) is fixed before test records are read.

Chapter 4

Implementation

4.1 Software, Hardware and Code Organisation

The system is written in Python with PyTorch 2.6 [18], TransformerLens 3.5.1 [13] and SAE-Lens 6.45.3 [14], and runs on a single laptop GPU (NVIDIA RTX 4050, 6 GB) under Windows 11. The first part of the project, the SAE edit and its Streamlit application with seven tabs, is unchanged and is described in the first report. The transfer study adds only new files: the library `src/transfer/` (translators, edits relayed through a map, the batched trainer, judges, Import) and the numbered command-line tools `00` to `27` in `tools/transfer/`, one per experimental step. Older functions are imported and never edited, so earlier results cannot change.

4.2 Reproducibility

Every tool prints its pass rule, saves its raw terminal output as a log, and writes a result file with per-run details (epoch curves, doses, seeds) and not only the final numbers. Result files and logs of every experiment are copied into a data pack under version control; all figures in this report are drawn from that pack alone by a script that runs no model. Larger local artefacts (trained maps, per-sentence results, caches; 2.6 GB) are not in git; they are packed with a manifest of file sizes and checksums, and the code commit they were made with, so that a restored copy can be verified. Pass bars were committed to the plan file before the corresponding tool was run, so the order of events can be checked in the repository history.

4.3 Engineering Notes

Batched training. The trainer of the map pads texts to one length and sends them through the remaining layers of medium together; because the model is causal, padding does not change the result (the per-text loss differs from the one-at-a-time code by 3×10^{-6}, the gradient cosine

is 1.000000), and training became about 14 times faster. **GPU memory.** The largest training set (7,550 texts padded to 57 tokens, 3.8 GB) nearly filled the 6 GB card; runs then slowed five- to ten-fold without any error (the varied-edits run took 6.2 hours instead of the one hour estimated). Training tensors are now kept off the GPU and time estimates are taken from the largest measured set. **A correction found by reading.** The "rewordings" figure of the random-word control was first compared with the real target's unchanged level instead of its own; this was found, recomputed, and the earlier claim withdrawn (Section 5.5).

4.4 Development Process

The software and the text of this report were developed with an AI coding assistant working under the authors' direction. The authors chose the research questions, the pass bars, the controls and the next experiment after each result, and ran or approved every long job; the assistant wrote and tested code, drafted documentation and logs, and was asked to record failures and corrections as carefully as successes. [TODO: confirm the wording of this statement with the project guide and the college policy]

Chapter 5

Results and Discussion

Unless stated otherwise, numbers are for the 88 primary test records and medium layer 16; "top-1" means that the edit's target becomes the model's first choice. Every pass bar was fixed before its run (Appendix A, Table A.1).

5.1 The Transient Edit in One Model

The first part of the project compared three editors on 300 held-out CounterFact prompts (Figure 5.1, Table 5.1). Per-feature gradient descent reached rank 1 on 271 prompts, against 161 for the Hybrid sweep given the same allowed range of 0 to 3 times per feature and 94 for the original sweep settings. It solved 110 prompts the wide sweep missed and lost none, had a lower side effect on the same prompt on 160 of the 161 prompts both solved, and took about 4 to 6.5 times less compute; part of the side-effect advantage is by construction, since the loss contains a side-effect term, and the median target probability at rank 1 is only 7.7%. Earlier sweep studies showed that the supply of usable features, not the depth of the intervention, is the limit: drawing candidates from all prompt positions brought the target to rank 1 on 10 of 13 comparable prompts against 4 of 13 for the last token, and removing the per-feature safety filter lost 10 of 29 successes.

Reaching rank 1 proved a weak test (Figure 5.2). An additive edit that switches on silent features pushed random unrelated words to rank 1 on 20 of 20 prompts (the multiplier edit: 1 of 20), at a median size of 126 to 192% of the residual norm against 39% for multipliers; on 20 unrelated prompts it changed the top word 24% of the time (multipliers: 6%) and its transfer to reworded prompts was as large for a random word as for the true answer. Reachability by steering is therefore not evidence about what a model knows, and an early classification of some facts as "absent from the model" was withdrawn. This is why every later result is judged against controls.

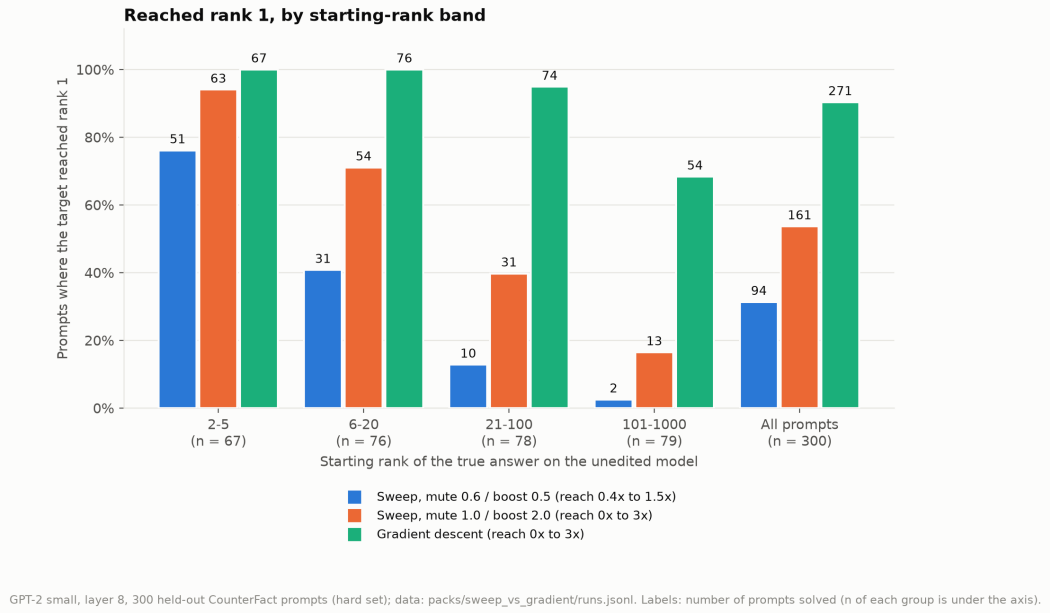


Figure 5.1: Prompts brought to rank 1 (labels give the count) by starting rank of the true answer, GPT-2 Small layer 8, 300 held-out CounterFact prompts. Gradient descent is the green bars.

Table 5.1: Single-model editing on 300 held-out CounterFact prompts.

Editor (allowed range per feature)	**Reached rank 1**	**Share**
Sweep, original settings (0.4 to 1.5 times)	94 of 300	31%
Sweep, wide range (0 to 3 times)	161 of 300	54%
Gradient descent (0 to 3 times)	**271 of 300**	**90%**

5.2 The Channel: Translator Quality and a Positive Control

A ridge map fitted on wikitext-2 keeps the receiver working (Table 5.2): stitching one model's first layers to the other's last layers through the map recovers 91% (small to medium) and 98% (medium to small) of the receiver's own quality; in the second case the stitched loss is 4.31 against 4.24 for small alone and 3.96 for medium. The map copies which word changed (cosine of the translated and true difference at the changed word 0.63 to 0.76 over eight layer pairs) but the summary at the last position only weakly (0.24 to 0.31); only the two medium to small pairs reached the 0.3 line fixed for the translator gate, so the translator passes that gate for Import and not for Export.

As a positive control we swap one country for another in a template sentence. The change is computed as a difference of the source model's states, translated, and added to the receiver, and counted as a hit when the receiver's top answer becomes the swapped capital. The translated change succeeds on 55% (Export) and 33% (Import) of test cases, against at most 0.4% for a random vector, a random map and another pair's difference (Figure 5.3). The control is mostly a channel check at the changed word's position; its test cases share countries and are not independent, and it is neither an edit nor a fact.

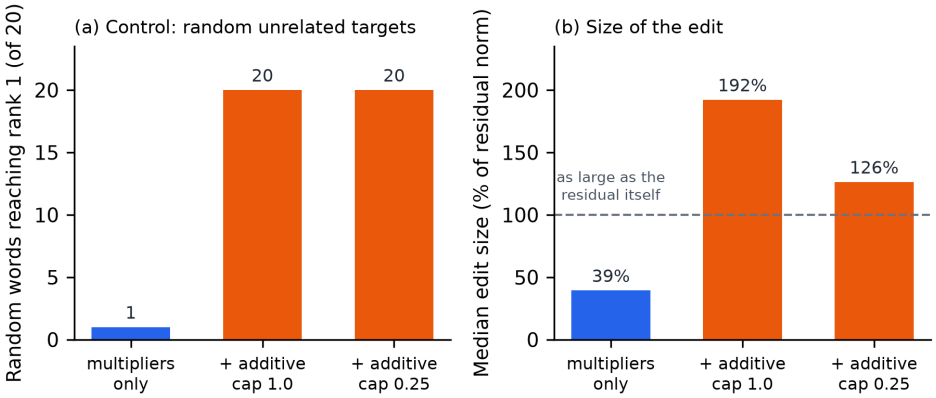


Figure 5.2: An additive edit forces random words to rank 1 (a) at sizes comparable to or larger than the residual stream itself (b); the multiplier edit does not.

Table 5.2: Ridge translators for the two directions studied (wikitext-2, held-out tokens).

Direction	R^2 **per dimension**	**Stitching recovers**	**Difference cosine at the last position**
small layer 8 → medium layer 16 (Export)	0.58	91%	0.28
medium layer 16 → small layer 8 (Import)	0.66	98%	0.31

5.3 Export with the Text-Fitted Map

For each of the 88 primary test records the edit tuned in small (200 multipliers, rank 1 in small) is relayed to medium through the ridge map, scaled by a dose chosen on dev, and added at layer 16 (Figure 5.4, Table 5.3). The effect is real, graded and specific. The target's median rank in medium falls from 142 to 14 (23 at layer 12), it becomes the top answer on 8% of records (5% at layer 12), reworded prompts favour the new answer on 52% against 36% unchanged, and neighbours lose only 4 points. A random vector and another record's edit do nothing (0% top-1, ranks 157 and 128). A plain word push along the target's output direction makes the target the top answer on 83% of records, but only 23% of neighbours keep the true answer, and stating the fact in medium's prompt lifts rewordings to 97% at the cost of 23 points of neighbours (Figure 5.5). The pass bar fixed beforehand (top-1 of at least 15% and 10 points above the strongest control) was not met. The edit moves medium toward the target without damaging its other answers, but it rarely does so cleanly enough to reach rank 1.

5.4 Searching for a Better Translator

Because the effect is real but weak, we tried seven changes, each with its pass rule written first (Table 5.4, Figure 5.6).

Layer and capacity. Injecting at layers 4, 8, 12, 16 and 20 gave 0, 5, 5, 8 and 1%; splitting

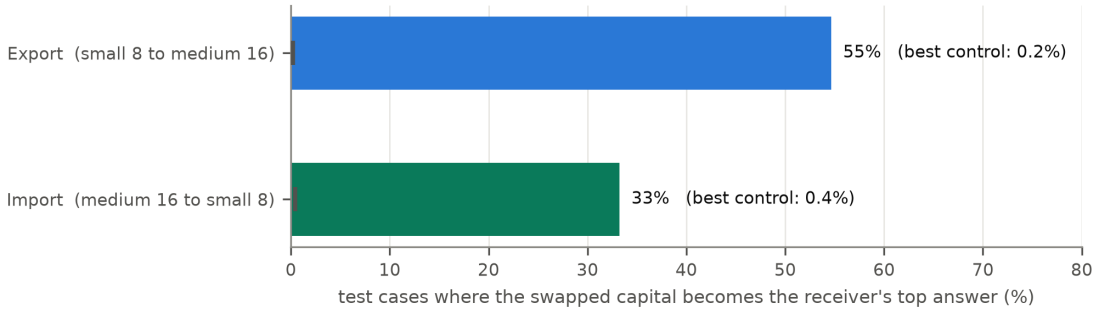


Figure 5.3: Country-swap control: the translated difference moves the receiver far above the best control (tick).

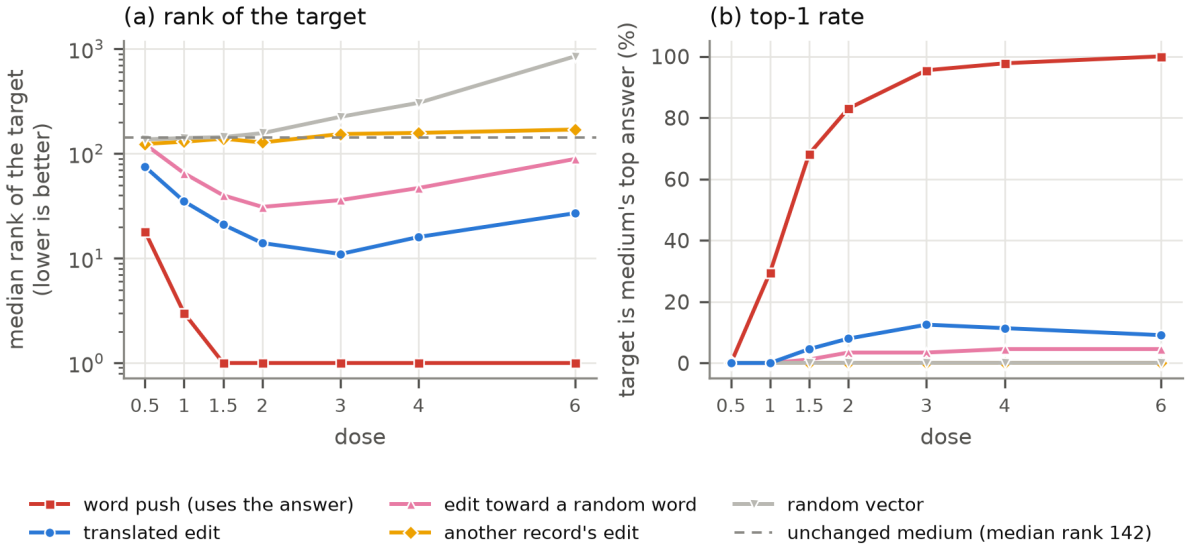


Figure 5.4: Export of the edit tuned in small, small layer 8 to medium layer 16, 88 test records: rank (a) and top-1 rate (b) against the dose. The word push uses the answer and is the comparison, not a method.

the dose over several layers did no better (best set 7%), so the layer is not the limit. A neural map fitted on the same text fits it much better (R^2 +0.09 to +0.11) but exports no better (8%): a better fit to ordinary text does not help.

A map trained on edits. Training the linear map so that medium's answers move as small's do (method 3 of Chapter 3) on 289 successful edits roughly doubled the result, from 8% to 18% top-1 and from median rank 14 to 7 (paired $p = 5 \times 10^{-9}$), still specific to the record (another record's edit: 0%). It did not reach the 50% bar. More training edits (25, 50, 100%: 13, 13, 20%), longer training and a direct rank-1 term in the loss (16 to 21% for every weight) left it at about 20%. One step produced a misleading figure: with 40 epochs the dose chosen on 28 dev records by top-1 was 1.0 and gave 2% against 20%; a post-hoc check showed that at dose 2.0 both lengths give 20%, and the dose is chosen by the dev log-rank gain from then on.

Why it plateaus. With the rank-1 term the map learned its 289 training targets (95% top-1 on them) but not unseen words (16%): it memorises. We then gave it 1,221 more edits

Table 5.3: Export of the edit through the ridge map (88 primary test records; dose chosen on dev). Unchanged medium: median rank 142.

Arm	Top-1	Median rank	Rewordings	Neighbours
Medium unchanged	0%	142	36%	67%
Translated edit	**8%**	**14**	**52%**	**63%**
Random vector	0%	157	38%	66%
Another record's edit	0%	128	36%	65%
Edit toward a random word (own word)	3%	31	–	–
Word push (uses the answer)	83%	1	82%	23%
Fact stated in medium's prompt	–	–	97%	44%
Small with the edit itself	–	–	67%	54%

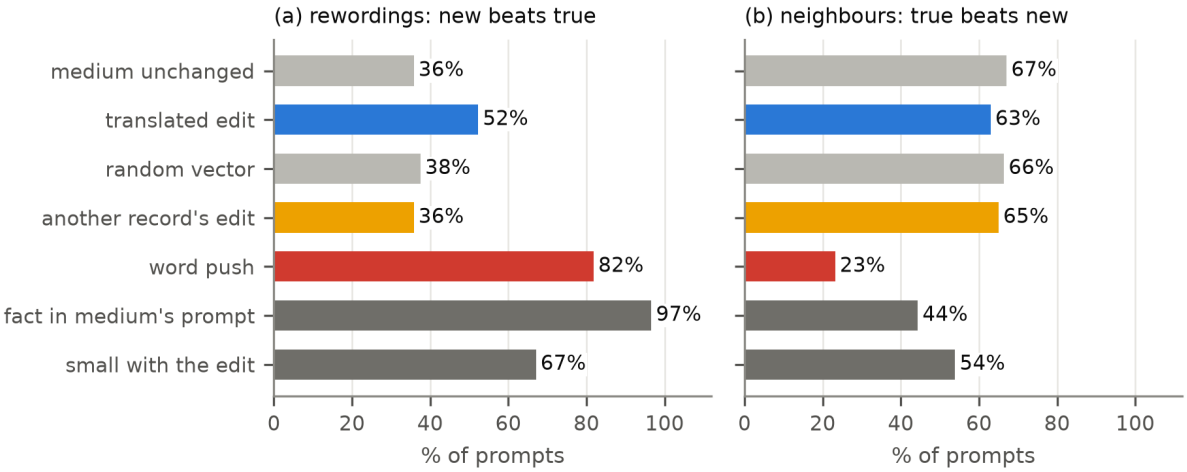


Figure 5.5: What the export does to reworded prompts and to neighbouring facts. Grey bars are references. The word push and the fact in the prompt damage neighbours far more than the translated edit.

toward 759 additional random target words (the 289 real edits cover only 116). Memorisation vanished (training minus test top-1: 12 points down to 3) but unseen records got *worse*, 20% to 14% (Figure 5.7); training top-1 fell as well while the training loss stayed the same, because the loss asks medium to copy small's shift, not to reach rank 1. Variety of target words was therefore not the limit.

Per-sentence tuning is a ceiling. Re-tuning the 200 multipliers against medium's output for each sentence reaches the top answer on 95% of records (84% on all 150), with the same neighbour cost as the source edit inside small (8 points against 9). The channel can carry a rank-1 edit; what fails is the compatibility of an edit tuned for small with medium. The solution found is only partly in the direction of the original edit (cosine 0.51 at the last position against 0.02 by chance, and 0.71 of its size), so no map that merely transforms the given edit can supply what is missing. This ceiling is not a transfer, since medium's output is used for every sentence, and it also reaches 83% for a random word.

Table 5.4: The search for a better translator (88 primary test records, same edits, controls and judging throughout).

Step	What changed	Top-1	Rank	Pass bar
D3	Ridge map fitted on text (baseline)	8%	14	fail
–	Neural map fitted on text	8%	14	no better
D5	Several injection layers	7%	11	fail
D7	Map trained on 289 edits	18%	7	fail
D8–D10	More data, longer training, rank-1 loss term	13–21%	7	fail
D13	+ 1,221 edits toward 759 new words	14%	9	fail
D6	Dials re-tuned per sentence (ceiling)	95%	1	neighbours fail

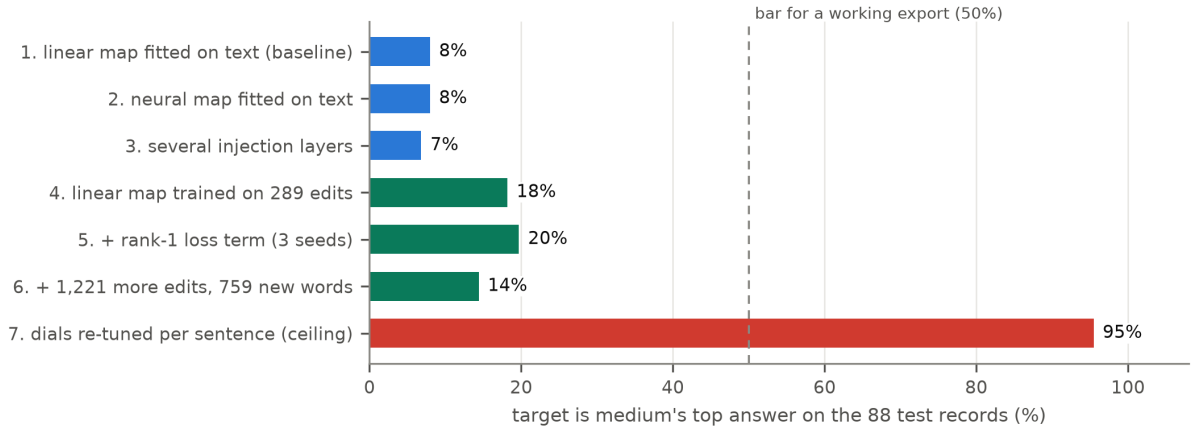


Figure 5.6: Top-1 of every export variant on the same 88 test records. Blue: map fitted on text; green: map trained on edits; red: per-sentence tuning against medium's output (a ceiling, not a transfer).

5.5 What Is Carried: a Fact or a Push?

For the edit inside small, a pre-stated rule gave "read-out steering": the gain on reworded prompts was 1.04 against 0.82 for a rank-matched random-word edit ($p = 0.032$), and an edit restricted to the subject's tokens succeeded on 12% of records against 59% for all positions; the evidence is weak either way. For Export, each arm's rewordings gain must be measured against its *own* unchanged level (a random word starts lower, at 25% against 36%). Doing so (Figure 5.8), D6's dials gain 24.4 points for the real target and 17.0 for a random word (difference +7.4, 95% interval 0.0 to +15.3, $p = 0.031$), and the map trained on edits gains 19.9 and 18.2 ($p = 0.36$); the random word also loses more neighbours (12.3 against 7.3 points). Neither difference passes the $p < 0.01$ line fixed beforehand. An earlier statement that the real target's gain was four times the random word's compared the wrong baselines and was withdrawn. What carries to reworded prompts is therefore a push toward the targeted word, not something specific to the fact.

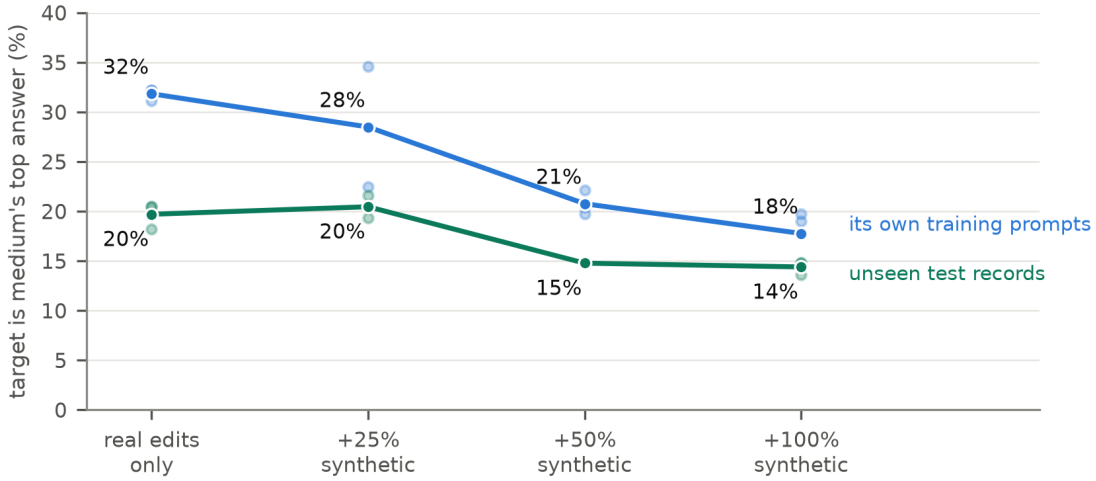


Figure 5.7: More and more varied training edits: top-1 on the map's own training prompts and on unseen test records (3 seeds, dots). The gap closes, but the test result does not rise.

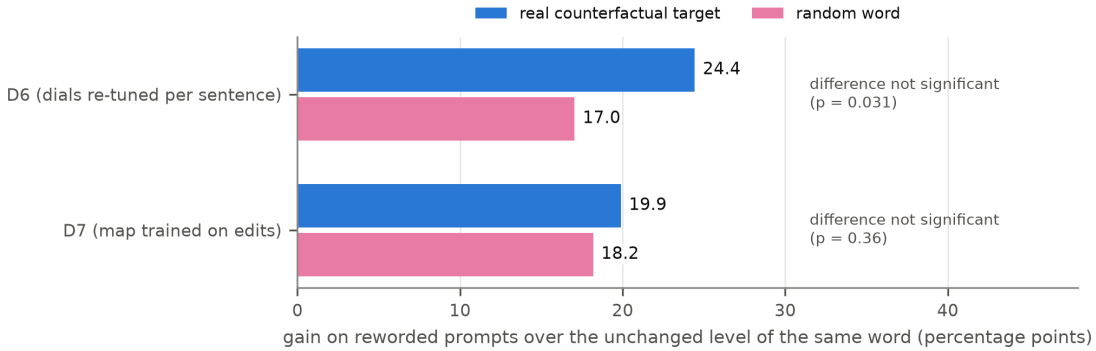


Figure 5.8: Gain on reworded prompts, each word against its own unchanged level: the real target and a random word gain about the same.

5.6 Does Small's SAE Matter for Steering Medium?

We repeated the per-sentence tuning with 200 random directions in place of the 200 SAE decoder directions (rows scaled to the SAE norms, the same firing pattern, loss and settings). The SAE directions reached 95% top-1 and the random ones 81% (62% against 83% for a random-word target), a difference of 15 points that meets the pre-set rule that the SAE basis "matters"; rewordings (60% against 58%) and neighbours (59% against 57%) are almost the same (Figure 5.9). The SAE makes the tuning more effective but is not what makes steering possible. The random basis still uses the SAE's activation pattern, so "no SAE at all" is untested, and one random draw was used.

5.7 Import: Medium's State into Small

Of the 21,789 facts, medium alone gets 9.9% right and small 8.3%; 780 facts are right in medium and wrong in small (for 71% of them the true answer is small's second to fifth choice).

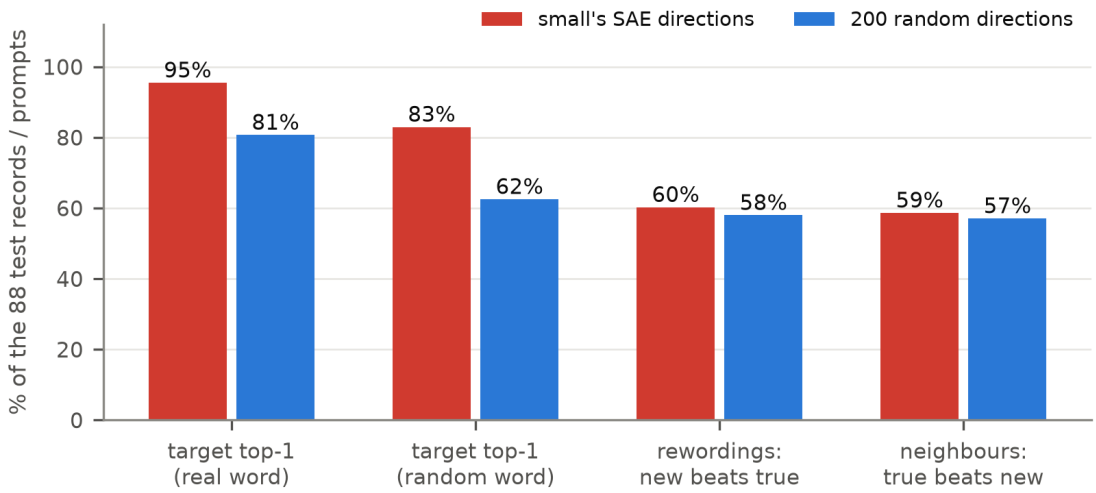


Figure 5.9: The tuning of Figure 5.6, step D6, with small's SAE directions and with 200 random directions (88 test records).

Table 5.5: Import on the held-out half B of the facts (10,895 facts): replacement and the 50/50 mix against the two models alone.

Measure	**Small**	**Medium**	**Replace** ($\gamma = 1$)	**Mix** ($\gamma = 0.5$)
Accuracy over all facts	8.5%	10.2%	7.4%	**9.5%**
Facts small got wrong, now right	–	–	31.9%	28.6%
Facts both knew, still right	100%	–	78.9%	98.3%
Facts only small knew, still right	100%	–	33.3%	80.5%
Agreement with medium's top answer	71.0%	100%	74.5%	76.2%

Full replacement of small's layer-8 state by medium's translated state rescues 262 of these 780 (33.6%; controls: wrong-sentence state 0.8%, average state 0%, random map 0%), but breaks 22% of the facts both models knew and 70% of those only small knew, so small's accuracy falls from 8.30% to 7.17%; a filter through small's SAE rescued only 6.3%. The pre-set bar (50% rescue, accuracy at least half-way to medium, 90% of known facts kept) was not met, although the controls show that real information crosses.

A **softer mix** was then tried, with its rule written before the build: $\gamma \in \{0.1, \ldots, 1\}$ and gating by medium's confidence, the setting chosen on one half of the facts and reported on the other (half B, 10,895 facts). Mixing in half of medium's state ($\gamma = 0.5$) raises small's accuracy from 8.5% to 9.5% (medium alone: 10.2%), gaining 162 facts and losing 53 ($p = 2.5 \times 10^{-14}$), keeps 98% of the facts both knew and 80% of those only small knew, and rescues 28.6% of the facts small gets wrong (Table 5.5, Figure 5.10); gating by confidence did not beat the plain mix. This meets the rule (gain of at least 0.5 points, 90% of known facts kept, $p < 0.01$) and is the first net-positive Import result. It is a small gain (one point), it needs medium to run on the same sentence, the old rescue bar of 50% is still not met, and the controls of the replacement test were not rerun for the mix.

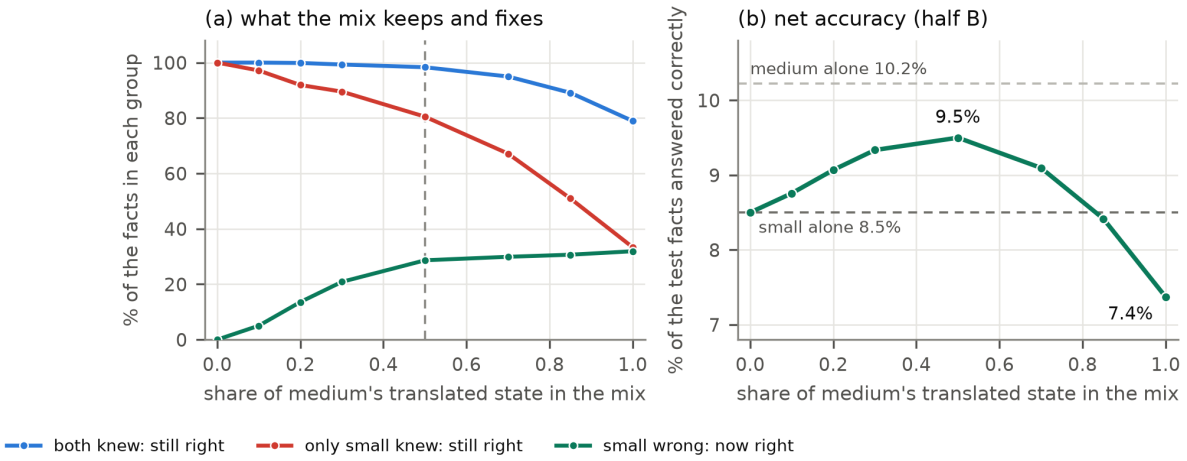


Figure 5.10: Import by mixing small's own state with medium's translated state. (a) What the mix keeps and fixes. (b) Net accuracy: best at a 50/50 mix, below small alone under full replacement.

5.8 Discussion and Threats to Validity

What the evidence supports. A linear channel between GPT-2 Small and Medium exists and keeps the receiver working. The change an edit makes in small moves medium's answers specifically, in a graded way, and without damaging its neighbours, but every translator that does not look at medium's output stops at about 20% top-1, far below the 50% bar; looking at medium for every sentence reaches 95%. Together with the controls this puts the limit on the *edit's compatibility*, not on the channel, the layer, the translator's capacity or the amount of data. What crosses is a push toward the targeted word and not a fact. In the other direction a half-and-half mix gives a small but statistically clear gain.

Threats to validity. (1) One model pair, one dataset (single-token CounterFact facts) and one SAE; nothing here says the results transfer to other models, layers or relations. (2) 88 primary test records, so one record is about one point, and training seeds share their data, so ranges understate the noise. (3) Choices of layer, dose and epoch rest on 28 to 50 dev records; the dose rule was changed once after a failure and the change is labelled post-hoc. (4) Several secondary results are descriptive diagnostics with their own rules, not method gates (the soft Import, the SAE control). (5) The additive edit, generated text, other model pairs and comparison with ROME and MEMIT were not tested. (6) The map follows earlier model-stitching work [37] and is not claimed as new.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This report started from a transient edit of GPT-2 Small through the features of a sparse autoencoder, which brought the true answer to rank 1 on 271 of 300 held-out prompts, and asked whether the change such an edit makes can be carried into another model at inference time. The answer has four parts.

1. **A channel exists.** A linear map fitted on ordinary text keeps GPT-2 Medium working and carries a known meaning change far above controls (55% and 33% against under 0.5%).
2. **Export carries a push, not a fact.** An edit tuned only in Small lifts the target's median rank in Medium from 142 to 14 specifically and without harming neighbours, but reaches rank 1 on only 8% of records. Seven attempts to improve it (other layers, a neural map, a map trained on edits, longer training, a rank-1 loss, more and more varied training edits) stop at about 20% at best; tuning the edit against Medium's output for each sentence reaches 95%, so the channel can carry a rank-1 edit and the limit is the edit's compatibility with the receiver. Reworded prompts move equally for a random target word, so what crosses is a push toward the word.
3. **Import works as a small mix.** Replacing Small's state by Medium's rescues one in three of the facts Small gets wrong but lowers its accuracy; a half-and-half mix raises accuracy on held-out facts from 8.5% to 9.5% while keeping 98% of the facts both models knew.
4. **The SAE helps but is not required.** Small's SAE directions raise the steering success of per-sentence tuning from 81% to 95% over random directions with the same activation pattern.

Every pass bar was fixed before its run, every failure is recorded with its evidence, and two earlier statements were corrected when a fairer baseline was applied.

6.2 Future Work

- **Import.** Rerun the controls of the replacement test for the mix, test it on reworded prompts and other relations, and train the Import map so that Small's answers become right (the analogue of the map trained on edits).
- **Export.** Learn the per-sentence solution instead of tuning it (an "amortised" version of the ceiling), or a map that depends on the sentence; a regularised or non-linear map trained on many edits was designed but not completed.
- **Generality.** Other relations and model pairs (for example Gemma 3 270M and 1B, which have SAEs on both sides), the additive edit, generated text, and larger donors.
- **Comparison.** Head-to-head comparison with ROME, MEMIT and in-context editing, and a check of the related-work claims against the full text of the stitching papers.

References

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.

[2] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 17359–17372.

[3] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[4] A. Gupta, A. Rao, and G. Anumanchipalli, "Model editing at scale leads to gradual and catastrophic forgetting," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024; arXiv:2401.07453.

[5] N. Elhage, T. Hume, C. Olsson, *et al.*, "Toy models of superposition," *Transformer Circuits Thread*, 2022.

[6] T. Bricken, A. Templeton, J. Batson, *et al.*, "Towards monosemanticity: Decomposing language models with dictionary learning," *Transformer Circuits Thread*, 2023.

[7] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse autoencoders find highly interpretable features in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[8] A. Templeton, T. Conerly, *et al.*, "Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet," *Transformer Circuits Thread*, 2024.

[9] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid, "Steering language models with activation engineering," *arXiv preprint arXiv:2308.10248*, 2023.

[10] N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner, "Steering Llama 2 via contrastive activation addition," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 15504–15522.

[11] A. Zou, L. Phan, S. Chen, *et al.*, "Representation engineering: A top-down approach to AI transparency," *arXiv preprint arXiv:2310.01405*, 2023.

[12] Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts, "AxBench: Steering LLMs? Even simple baselines outperform sparse autoencoders," in *Proc. International Conference on Machine Learning (ICML)*, 2025; arXiv:2501.17148.

[13] N. Nanda and J. Bloom, "TransformerLens," GitHub repository, 2022. [Online]. Available: https://github.com/TransformerLensOrg/TransformerLens

[14] J. Bloom, C. Tigges, A. Duong, and D. Chanin, "SAELens," GitHub repository, 2024. [Online]. Available: https://github.com/decoderesearch/SAELens

[15] J. Bloom, "Open source sparse autoencoders for all residual stream layers of GPT2 small," *AI Alignment Forum*, 2024.

[16] T. Lieberum, S. Rajamanoharan, A. Conmy, *et al.*, "Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2," *arXiv preprint arXiv:2408.05147*, 2024.

[17] J. Lin and J. Bloom, "Neuronpedia: Interactive reference and tooling for analyzing neural networks," 2023. [Online]. Available: https://www.neuronpedia.org

[18] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[19] D. Arad, A. Mueller, and Y. Belinkov, "SAEs are good for steering – if you select the right features," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025; arXiv:2505.20063.

[20] L. Smith, S. Rajamanoharan, A. Conmy, C. McDougall, T. Lieberum, J. Kramár, R. Shah, and N. Nanda, "Negative results for SAEs on downstream tasks and deprioritising SAE research (GDM mech interp team progress update #2)," *LessWrong*, Mar. 2025. [Online]. Available: https://www.lesswrong.com/posts/4uXCAJNuPKtKBsi28/negative-results-for-saes-on-downstream-tasks

[21] M. Chaudhary and A. Geiger, "Evaluating open-source sparse autoencoders on disentangling factual knowledge in GPT-2 small," *arXiv preprint arXiv:2409.04478*, 2024.

[22] M. Cui, L. Shen, and X. Yang, "SAE interventions are unreliable: Post-intervention recovery of suppressed behavior," *arXiv preprint arXiv:2606.18322*, 2026.

[23] E. Farrell, Y.-T. Lau, and A. Conmy, "Applying sparse autoencoders to unlearn knowledge in language models," *arXiv preprint arXiv:2410.19278*, 2024.

[24] S. Chalnev, M. Siu, and A. Conmy, "Improving steering vectors by targeting sparse autoencoder features," *arXiv preprint arXiv:2411.02193*, 2024.

[25] Y. Zhao, A. Devoto, G. Hong, X. Du, A. P. Gema, H. Wang, X. He, K.-F. Wong, and P. Minervini, "Steering knowledge selection behaviours in LLMs via SAE-based representation engineering," in *Proc. NAACL*, 2025; arXiv:2410.15999.

[26] P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun, "Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[27] C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang, "Can we edit factual knowledge by in-context learning?" in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023; arXiv:2305.12740.

[28] P. Wang, N. Zhang, B. Tian, *et al.*, "EasyEdit: An easy-to-use knowledge editing framework for large language models," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024; arXiv:2308.07269.

[29] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller, "Sparse feature circuits: Discovering and editing interpretable causal graphs in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2025; arXiv:2403.19647.

[30] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[31] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[32] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[33] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[34] K. Lenc and A. Vedaldi, "Understanding image representations by measuring their equivariance and equivalence," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[35] Y. Bansal, P. Nakkiran, and B. Barak, "Revisiting model stitching to compare neural representations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[36] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proc. International Conference on Machine Learning (ICML)*, 2019.

[37] Chen *et al.*, "Transferring linear features across language models with model stitching," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025; arXiv:2506.06609.

[38] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

Appendices

Appendix A: Pass Bars and Verdicts

Every bar was written into the project plan, and committed, before the run that judged it; none was moved afterwards. "Edit's own" means the effect that the source edit has inside small itself. Verdicts are as printed by the tools.

Table A.1: Pass bars ("gates") and their verdicts.

Step	**Bar fixed before the run**	**Result**	**Verdict**
Gate 1 (A2)	Translator: difference cosine at the last position at least 0.3	Import pairs 0.31; Export pair 0.28	Import only
Gate 2 (B1)	Country swap beats the strongest control by a margin, paired test	55% and 33% against at most 0.4%	pass
Gate 3 (D)	Export top-1 at least 15% and 10 points above the strongest control	8%; margin 3 points	fail
Gate 4 (D5)	Best layer set at least 10 points above the best single layer, $p < 0.01$, top-1 at least 15%	7% against 8%; $p = 0.016$	fail
Gate 5 (D6)	Top-1 $\geq$ 50%; rewordings lift $\geq$ 80% of the edit's own; neighbours fall $\leq$ 5 points	95%; 0.91; 8 points	fail (neighbours)
Gate 6 (D7)	Top-1 $\geq$ 50%; rewordings lift $\geq$ 0.8; neighbours fall no more than the edit's own	18%; 0.74; 7 points	fail
Gate 7 (D10)	As Gate 6, plus real target 10 points above the random word	20%; 0.74; met; met	fail
Gate 8 (Import)	Rescue $\geq$ 50% of the 780 facts; accuracy halfway to medium; 10 points above controls ($p < 0.01$); 90% of known facts kept	33.6%; 7.2% against 9.1%; met; 78.3%	fail
Gate 9 (D13)	Gate 7 unchanged, on the full varied training set	14%; 0.62; met; met	fail
Rule E5	Soft Import: gain $\geq$ 0.5 points on held-out facts, 90% kept, $p < 0.01$	+1.0 point; 98.3%; $p = 2.5 \times 10^{-14}$	met

Step	Bar fixed before the run	Result	Verdict
Rule S1	SAE basis "matters" if top-1 is higher by 10 points or rewordings lift by 5	+15 points; +2 points	matters
Rules S2, S3	Real target's rewordings lift larger than a random word's, $p < 0.01$	$p = 0.031$ (D6); $p = 0.36$ (D7)	not shown

Two notes written after the fact do not change any verdict: the 5-point neighbour line of Gate 5 was stricter than the source edit's own loss of 9 points inside small, so Gates 6, 7 and 9 use the edit's own loss; and the dose rule (dev top-1 on 28 records) was replaced by the dev log-rank gain from step D10 on, after step D9b showed it to be fragile.

Appendix B: Reproducing the Results

From the repository root on branch `cross-model-transfer`, with `HF_HUB_OFFLINE=1` once the models are cached. Run times are for the RTX 4050 laptop GPU.

```
.venv\Scripts\python.exe tools/transfer/01_fit_maps.py          ridge
   translators, 138 s
.venv\Scripts\python.exe tools/transfer/03_swap_control.py      country
   swap, 279 s
.venv\Scripts\python.exe tools/transfer/05_export_edits.py      edits in
   small, 1,263 s
.venv\Scripts\python.exe tools/transfer/06_export_eval.py       controlled
   export, about 54 min
.venv\Scripts\python.exe tools/transfer/10_multilayer_export.py   several
   layers, 1,113 s
.venv\Scripts\python.exe tools/transfer/11_b_aware_edit.py      per-
   sentence tuning (ceiling), 2,149 s
.venv\Scripts\python.exe tools/transfer/12_output_matching.py  map trained
    on edits, 3,896 s
.venv\Scripts\python.exe tools/transfer/16_rank1_term.py        rank-1 loss
    term, 1,959 s
.venv\Scripts\python.exe tools/transfer/21_varied_edits.py      extra
   training edits, 3,542 s
.venv\Scripts\python.exe tools/transfer/22_varied_training.py  varied-
   edits training
.venv\Scripts\python.exe tools/transfer/19_import_experiment.py   Import
   by replacement, 249 s
.venv\Scripts\python.exe tools/transfer/23_softer_import.py    soft Import
   , 304 s
.venv\Scripts\python.exe tools/transfer/24_sae_basis_control.py   random-
   basis control, 1,677 s
.venv\Scripts\python.exe tools/transfer/26_d6_rewordings_test.py
   rewordings test, 221 s
```

```
.venv\Scripts\python.exe tools/transfer/27_random_word_baseline.py   same
   for a trained map, 240 s
.venv\Scripts\python.exe tools/make_report_v2_figures.py       report
   figures from the data pack
```

All pass bars and readings are in `docs/cross_model_transfer/PLAN.md`, the story of each step in the research journal (entries 31 to 37), and every result file and raw log in `docs/Research_Journal/packs/cross_model_transfer/`. The large local artefacts are described by a manifest in the same folder.